DESARROLLO DE API REST Y APLICACIÓN WEB PARA LA GESTIÓN DE USUARIOS Y SERVICIOS EN UN NEGOCIO DE MOTOCICLETAS KTM

Julian Gacha
Técnico en Programación de Software – 3236085
Evidencia: Aplicación web funcional. GA4-220501096-AA1-EV02

Resumen

En este proyecto se desarrolló una aplicación web funcional para la gestión de usuarios y servicios en un negocio de motocicletas KTM. Se implementó un backend con Spring Boot y una base de datos MySQL, junto con un frontend en React que consume las API.

Palabras clave

API REST, Spring Boot, React, MySQL, CRUD, Autenticación.

I. INTRODUCCIÓN

Se desarrolló una solución completa cliente-servidor integrando backend y frontend, permitiendo la gestión de usuarios y servicios.

II. DESARROLLO DEL BACKEND

A. Configuración del proyecto
Se creó un proyecto en Spring Boot con dependencias necesarias.

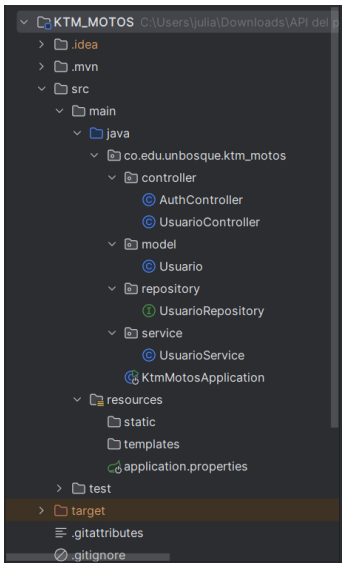

Fig. 1. Creación del proyecto en Spring Boot.

B. Conexión a base de datos
Se configuró MySQL en application.properties.

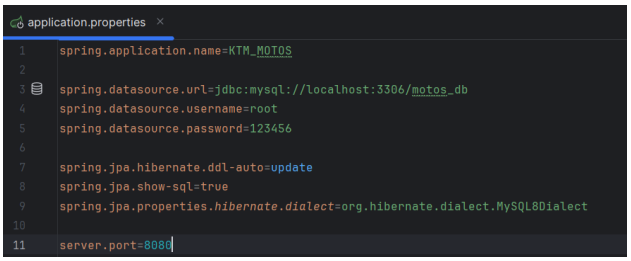

Fig. 2. Configuración de conexión a MySQL.

C. Implementación de API REST
Se desarrollaron endpoints CRUD.

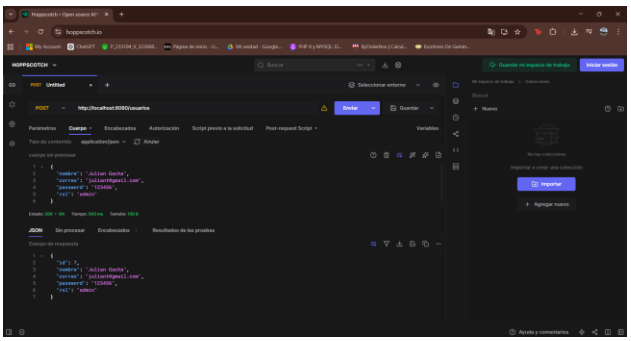

Fig. 3. Prueba de API en Postman.

III. DESARROLLO DEL FRONTEND

A. Configuración Se utilizó React con Vite.

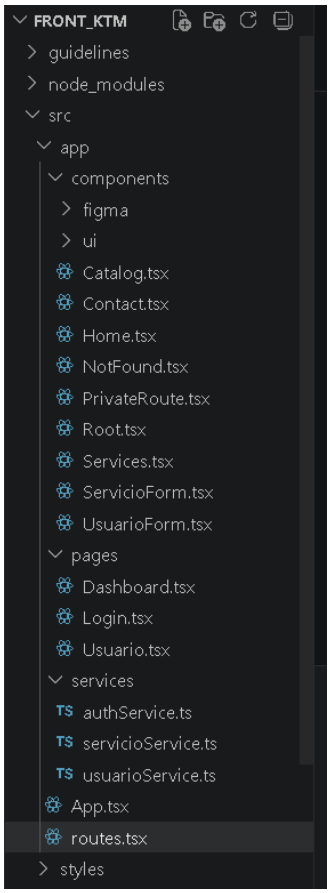

Fig. 5. Estructura del proyecto React.

B. Consumo de API
Se utilizó Axios para comunicación.

```
src > app > services > TS authService.ts > ...
import axios from "axios";

const API = "http://localhost:8080"; // tu backend

export const loginRequest = (data: {
  correo: string;
  password: string;
}) => {
  return axios.post(`${API}/auth/login`, data);
};
```

Fig. 6. Consumo de API desde React.

C. Autenticación
Se desarrolló formulario de login.

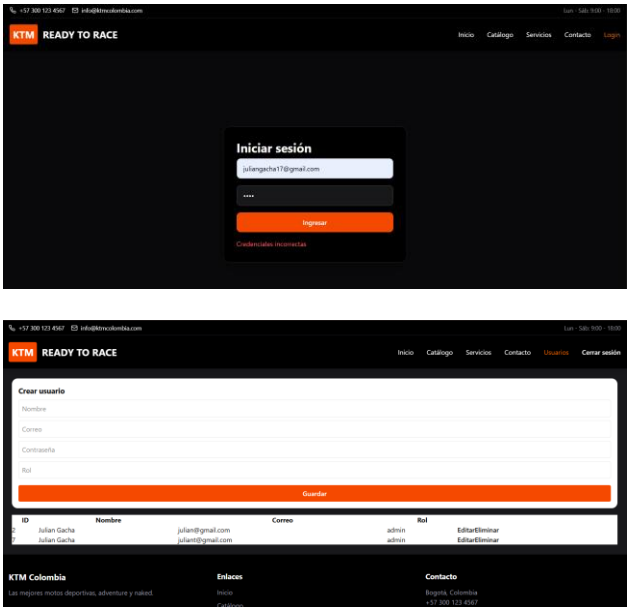

Fig. 7. Interfaz de login.

D. Protección de rutas
Se implementó PrivateRoute.

```
src > app > components > PrivateRoute.tsx > PrivateRoute
import { Navigate } from "react-router-dom";

interface PrivateRouteProps {
  children: React.ReactNode;
}

export default function PrivateRoute({ children }: PrivateRouteProps) {
  const usuarioGuardado = localStorage.getItem("usuario");

  if (!usuarioGuardado) {
    return <Navigate to="/login" replace />;
  }

  return <>{children}</>;
}
```

Fig. 8. Protección de rutas.

E. CRUD usuarios
Se implementó gestión completa.

```
src > app > services > TS usuarioService.ts > ...
import axios from "axios";

const API = "http://localhost:8080/usuarios";

export interface Usuario {
  id?: number;
  nombre: string;
  correo: string;
  password: string;
  rol: string;
}

export const getUsuarios = () => axios.get<Usuario[]>(API);

export const createUsuario = (usuario: Usuario) => axios.post(API, usuario);

export const updateUsuario = (id: number, usuario: Usuario) =>
  axios.put(`${API}/${id}`, usuario);

export const deleteUsuario = (id: number) => axios.delete(`${API}/${id}`);
```

Fig. 9. CRUD de usuarios.

IV. PRUEBAS

Se realizaron pruebas con Postman y navegador validando el correcto funcionamiento del sistema.

[INSERTAR FIGURA 10 AQUÍ]
Fig. 10. Pruebas finales del sistema.

V. RESULTADOS

Se logró la integración completa entre backend y frontend, cumpliendo con los requisitos del sistema.

VI. CONCLUSIONES

El proyecto demuestra la correcta implementación de una arquitectura web moderna con API REST y frontend interactivo.

VII. REFERENCIAS

[1] Spring Boot Documentation.
[2] React Documentation.
[3] MySQL Documentation.
[4] Axios Documentation.